ALLIANCE UNIVERSITY

Alliance School of Advanced Computing

MICRO-PROJECT REPORT

(Report after Execution)



Title of Micro-project

INVENTORY MANAGEMENT AND SALES VISUALIZATION SYSTEM

Using Python, Tkinter, SQLite, Pandas and Matplotlib

Course Code	: E1CSA348
Course Title	: Advanced Python Programming
Name of Course In-charge	: Dr.CHETAN J SHELKE
Academic Submission	: End-Semester Micro-project

Student Team Members

1. B Hithesh Reddy
2. Shaik Kishwar Sahil
3. M Mahammad Asif

INDEX

Sl. No.	Particulars	Page No.
–	Project Overview	3
1	Rationale	4
2	Competency of the Course	4
3	Course Outcomes Addressed	4
4	Information Collection	4
5	Theory and Design of the Micro Project	5
6	Methodology Followed	7
7	Outputs of the Micro–project (incl. GUI Screenshot)	8
8	Benefits of this Micro–project	10
9	Area of Future Improvement	11
10	References	11

PROJECT OVERVIEW

The Inventory Management and Sales Visualization System is a Python desktop application for maintaining product inventory, recording sales transactions, and generating basic sales analytics. The final implementation uses Tkinter for the graphical user interface, SQLite for persistent local storage, Pandas for tabular analysis, and Matplotlib for visualization. The source code contains two main classes, Inventory and App, which separate database operations from GUI and user-interaction logic.

The application supports product entry through fields for name, category, price and stock. Stored products are displayed in a Tkinter Treeview. A user can record a sale by entering a Product ID and quantity. The program checks that the product exists and that sufficient stock is available, calculates the sale amount, stores the transaction, and reduces the stock value. The analytics function retrieves sales records into a Pandas DataFrame, calculates total revenue and the best-selling product, and creates a category-wise sales bar chart. The technologies used are summarised in Section 5.2 (Table 2).

TECHNOLOGY STACK

Technology	Purpose
Python	Core application logic, classes, functions and exception handling
Tkinter	GUI window, labels, entries, buttons, Treeview and message boxes
SQLite3	Persistent storage in inventory.db; products and sales tables
Pandas	DataFrame creation, sum(), groupby() and best-product analysis
Matplotlib	Category-wise sales bar chart

1. Rationale

Small shops, stationery stores and campus outlets still record stock and sales in registers or loose spreadsheets. This manual approach is time-consuming, error-prone and gives no quick view of which products sell best. When a sale is made, stock must be reduced by hand, totals must be calculated again and again, and summaries are produced only with considerable effort.

This micro-project addresses the problem by developing a compact desktop application, the Inventory Management and Sales Visualization System. The application stores product and sales records permanently in an SQLite database, provides a simple Tkinter interface for entering data, automatically reduces stock when a sale is recorded, and uses Pandas and Matplotlib to report total revenue, the best-selling product and category-wise sales. The project also lets the team apply object-oriented programming, database handling, exception handling and data analysis in one practical, working solution.

2. Competency of the Course

Develop modular, object-oriented Python applications that integrate a graphical user interface, a persistent database and data-analysis libraries to solve a practical problem, and present the resulting insights through meaningful visualizations.

3. Course Outcomes Addressed in this Micro Project

i. Apply object-oriented programming concepts (classes, objects, methods and encapsulation) to structure a complete Python application.

ii. Design and build a graphical user interface using the Tkinter library with event-driven programming.

iii. Create, store and retrieve data using SQLite through the sqlite3 module with parameterized queries.

iv. Analyze data using Pandas (DataFrame, groupby, aggregation) and visualize it using Matplotlib.

v. Handle runtime errors and invalid user input using exception handling and input validation.

4. Information Collection

Before development, the team studied how small retail businesses manage stock and sales, and listed the functions a basic inventory tool must provide: adding products with a name, category, price and quantity; viewing the current stock; recording a sale; preventing a sale when stock is insufficient; and summarizing sales. The following technical information was then collected from the official Python documentation, course material and tutorials.

- **Tkinter:** widgets such as Label, Entry, Button, Frame and ttk.Treeview, the grid and pack layout managers, and messagebox dialogs for user feedback.
- **SQLite (sqlite3):** a serverless, file-based relational database bundled with Python. Tables, primary keys with AUTOINCREMENT, parameterized queries (to avoid SQL injection) and commit operations were studied.
- **Pandas:** creating a DataFrame from database rows, and using groupby(), sum() and idxmax() to compute totals and find the best-selling product.
- **Matplotlib:** plotting bar charts from grouped data, with titles, axis labels and tight_layout().

- **Object-oriented design:** separating data-handling logic (Inventory class) from the user interface (App class) for clarity and reuse.

From this study, SQLite was chosen over text or CSV files because it gives reliable persistent storage, structured queries and no installation overhead. Tkinter was chosen because it ships with Python and is sufficient for a lightweight desktop interface.

The information required was grouped into two sets. Product information comprises the product ID, name, category, price and current stock; it identifies an item, supports the calculation of a sale and tracks the quantity available. Sales information comprises the product name, category, quantity and total amount of every recorded sale; the total is price × quantity, and the products table is updated separately so that it always shows the remaining stock.

Input validation was also considered. Price and stock are converted to numeric values when a product is added, and Product ID and quantity are converted to integers when a sale is recorded. The sell_product() method checks whether the product exists and whether the requested quantity exceeds the available stock, and error messages are shown through Tkinter message boxes when an operation cannot be completed.

Information Flow

Data moves through the system in five stages, as summarised below.

Stage	Component	Role
1	User Input	Product fields or sale fields entered in Tkinter
2	Application Logic	App class validates input and calls Inventory methods
3	Database	SQLite stores products and completed sales
4	Analysis	Pandas calculates revenue and grouped sales
5	Visualization	Matplotlib displays category-wise sales

Table 1: Information flow

5. Theory and Design of the Micro Project

5.1 Problem Statement and Objectives

Manual stock tracking is time-consuming, repeated sales calculations are tedious, best-selling products are hard to identify quickly, and raw figures are difficult to interpret without visualization. The objectives were to:

- develop a simple inventory management application;
- store product and sales information using SQLite;
- provide a user-friendly Tkinter interface;
- update stock automatically after every sale; and
- analyze and visualize sales data using Pandas and Matplotlib.

5.2 Technologies Used

Technology	Purpose
Python	Main programming language
Tkinter	Graphical user interface
SQLite	Database and persistent storage
Pandas	Data analysis

Technology	Purpose
Matplotlib	Data visualization
OOP	Program structure (Inventory and App classes)

Table 2: Technologies used

5.3 System Architecture

The system follows a layered design. The user interacts with the Tkinter GUI, which calls the Inventory class for application logic. The Inventory class reads and writes the SQLite database. Sales records are then passed to Pandas for analysis and to Matplotlib for visualization.

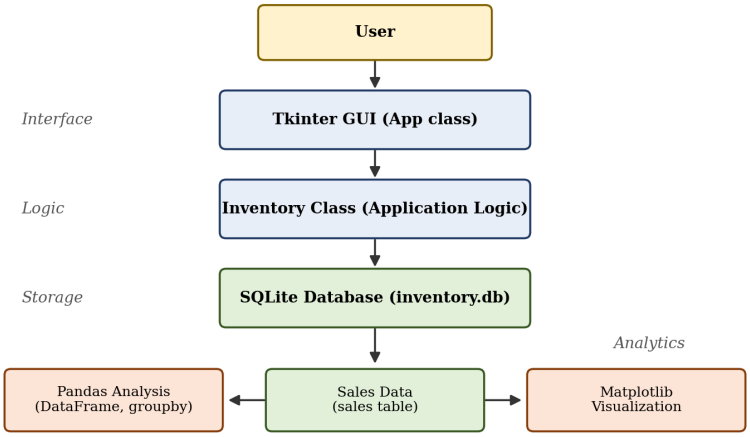


Figure 1: System architecture

5.4 Database Design

SQLite stores data in a local file, inventory.db, and two tables are created automatically on first run.

products table	Data Type	Description
id	INTEGER	Primary key, auto-incremented unique product ID
name	TEXT	Product name
category	TEXT	Product category (used for grouping)
price	REAL	Unit price in rupees
stock	INTEGER	Quantity currently available

Table 3: Structure of the products table

sales table	Data Type	Description
product	TEXT	Name of the product sold
category	TEXT	Category of the product sold
quantity	INTEGER	Number of units sold
total	REAL	Sale amount = price × quantity

Table 4: Structure of the sales table

5.5 Class Design (OOP)

Class	Responsibilities
Inventory	Handles the database connection, table creation, and product and sales operations through the methods create_tables(), add_product(), get_products(), sell_product() and get_sales().
App	Builds the Tkinter window, takes user input, and connects the interface to the Inventory class through the methods add(), load(), sell() and analytics().

Table 5: Classes and their responsibilities

Other concepts used include functions and methods, exception handling (try/except ValueError for invalid input), parameterized SQLite queries, the Pandas DataFrame and groupby().

5.6 Processing Logic

- **Add product:** collect name, category, price and stock; insert the record into products; commit the transaction; refresh the Treeview.
- **Display inventory:** retrieve all rows from products and insert them into the GUI table.
- **Record sale:** find the product by ID, check that it exists and compare the requested quantity with current stock.
- **Calculate sale:** total = price × quantity.
- **Update database:** insert the completed sale into sales and reduce the product stock in products.
- **Analytics:** load sales into a DataFrame, sum Total for revenue, group Quantity by Product for the best-selling product, and group Total by Category for visualization.

5.7 Exception Handling

The source code uses try-except blocks in the add() and sell() methods of the App class. In add(), the price is converted to float and the stock to int. In sell(), the product ID and quantity are converted to int. If the user enters invalid numeric data, a ValueError is caught and an error message is displayed. The database method sell_product() separately returns controlled messages for a missing product or insufficient stock. This prevents common input errors from crashing the program.

6. Methodology Followed

The team followed a step-by-step approach, completing and testing each stage before moving to the next.

1. **Problem identification:** the difficulties of manual stock and sales records were identified and the scope was kept to a simple, single-user desktop tool.
2. **Requirement analysis and planning:** the features, technologies, classes and the two database tables were decided.
3. **Database layer:** the Inventory class was written first, with automatic table creation and methods for adding products, listing products, recording sales and listing sales.
4. **Interface layer:** the App class was built with entry fields, buttons and a Treeview table showing products.
5. **Sales logic:** sell_product() checks that the product exists and that stock is sufficient, then inserts a sales record and reduces stock in one transaction.

6. **Analytics:** sales data was loaded into a Pandas DataFrame to compute total revenue and the best-selling product, and a category-wise bar chart was produced with Matplotlib.
7. **Testing and validation:** valid and invalid inputs were tried, and the results were noted (see Section 7.0).
8. **Documentation and presentation:** the report and the seminar presentation were prepared.

Core Logic of a Sale

The following extract shows how a sale is validated and how stock is updated automatically:

```
product = cur.execute(
    "SELECT name,category,price,stock FROM products WHERE id=?", (pid,)
).fetchone()
if not product:      return False, "Product not found"
if qty > stock:      return False, "Not enough stock"
total = price * qty
cur.execute("INSERT INTO sales VALUES(?,?,?,?)", (name, category, qty, total))
cur.execute("UPDATE products SET stock=stock-? WHERE id=?", (qty, pid))
self.con.commit()
```

Listing 1: sell_product() in the Inventory class

Analytics Logic

```
df = pd.DataFrame(sales, columns=["Product","Category","Quantity","Total"])
revenue      = df["Total"].sum()
best_product = df.groupby("Product")["Quantity"].sum().idxmax()
df.groupby("Category")["Total"].sum().plot(kind="bar", title="Category-wise Sales")
```

Listing 2: Analytics in the App class

Application Workflow

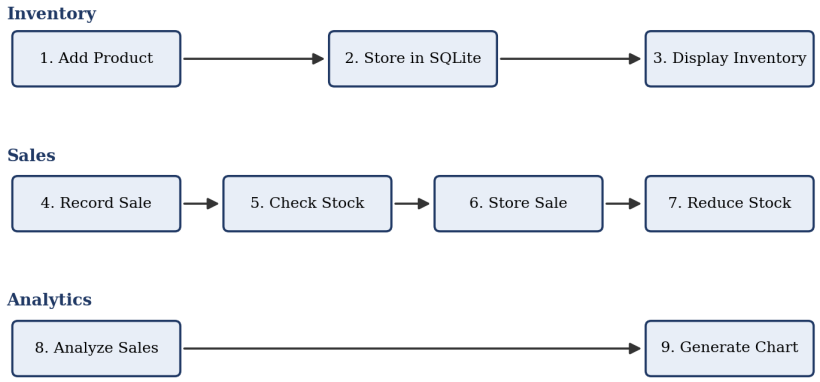


Figure 2: Application workflow (Inventory → Sales → Analytics)

Environment Setup

The application was implemented in Python. Tkinter is used for the desktop interface and sqlite3 for the local database, while Pandas and Matplotlib are used for analytics and visualization. The source file imports tkinter, ttk, messagebox, sqlite3, pandas and matplotlib.pyplot.

Testing

Testing covered successful product insertion, inventory display, successful sales, stock reduction, invalid numeric input, missing products and insufficient stock. The GUI screenshot in Section 7.1 was produced by executing the final source code (Appendix A) with sample records in the SQLite database: Laptop, Mouse, Notebook and Keyboard.

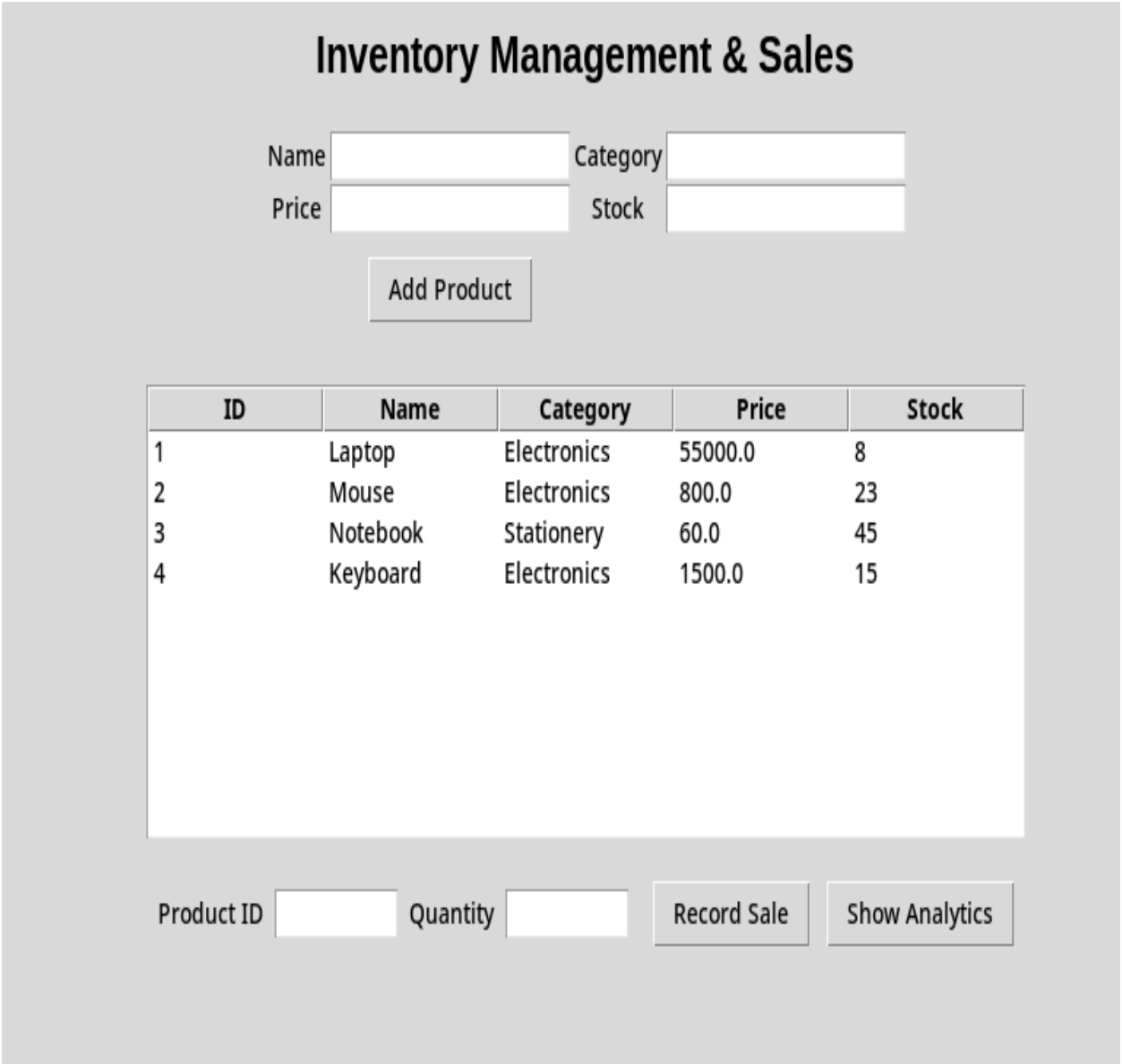
7. Outputs of the Micro–project

The completed application opens a window titled “Inventory Management System” containing the following parts, from top to bottom: a product-entry form (Name, Category, Price, Stock) with an Add Product button; a table listing all products with their ID, Name, Category, Price and Stock; and a sales panel with Product ID and Quantity fields and the Record Sale and Show Analytics buttons.

7.1 Prototype Output and GUI Screenshot

The prototype provides a complete desktop interface titled “Inventory Management & Sales”. The top portion contains the product input fields for Name, Category, Price and Stock, followed by the Add Product button. The central area contains a Treeview with ID, Name, Category, Price and Stock columns. The lower section contains the Product ID and Quantity inputs and the Record Sale and Show Analytics buttons. Figure 3 shows the interface generated by running the final code.

Figure 3: GUI output generated by executing the final Python code with sample inventory records



For this screenshot the database was populated with four sample products – Laptop, Mouse, Notebook and Keyboard – and the interface displayed them with their categories, prices and remaining stock. The functions of the application and their observed output are summarised below.

Function	Input	Observed / Expected Output
Product Addition	Name, Category, Price, Stock	Product stored in SQLite and shown in Treeview
Inventory Display	Stored product records	ID, name, category, price and stock displayed
Sale Recording	Product ID and Quantity	Sale stored and stock reduced
Sales Analytics	Stored sales records	Total revenue and best-selling product shown
Visualization	Category-wise sales totals	Matplotlib bar chart generated

Table 6: Functions and their output

7.2 Functional Output

No.	Test Case	Expected Output
1	Add a product with valid details	Product is saved and appears in the product table; message "Product added" is shown.
2	Add a product with non-numeric price or stock	Error message "Enter valid price and stock"; nothing is saved.
3	Record a sale with valid ID and quantity within stock	Sale amount (price × quantity) is displayed in ₹; stock in the table decreases.
4	Record a sale with quantity greater than stock	Error message "Not enough stock"; stock is unchanged.
5	Record a sale with an ID that does not exist	Error message "Product not found".
6	Record a sale with blank or non-numeric values	Error message "Enter valid values".
7	Show Analytics with no sales	Message "No sales data available".
8	Show Analytics after sales are recorded	Dialog with Total Revenue and Best Selling Product, followed by a category-wise bar chart.

Table 7: Test cases and expected outputs

7.3 Analytics Output

Show Analytics displays the total revenue (the sum of the Total column) and the best-selling product (the product with the highest total quantity sold), and then plots the sales of each category as a bar chart. Figure 4 shows the format of this chart using sample data.

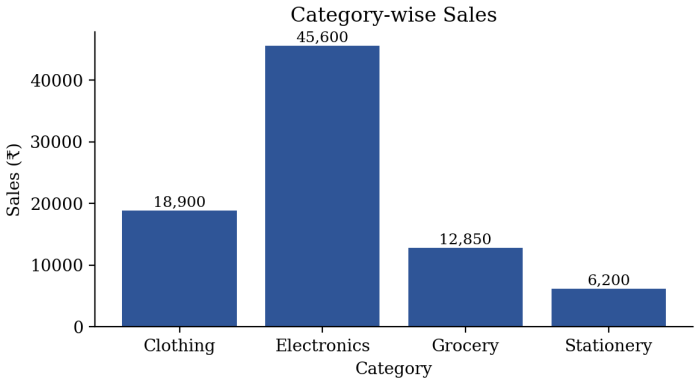


Figure 4: Category-wise sales chart (sample data used for illustration)

The analytics function produces three results from the stored sales records, each obtained with a single Pandas operation.

Metric	Code Logic	Meaning
Total Revenue	`df['Total'].sum()`	Overall value of recorded sales
Best-Selling Product	`groupby('Product')['Quantity'].sum().idxmax()`	Product with highest units sold
Category Sales	`groupby('Category')['Total'].sum()`	Revenue summarized by category

Table 8: Analytics metrics and code logic

7.4 Findings, Conclusion and Limitations

- Products can be stored permanently and displayed in the table.
- Sales can be recorded and stock decreases automatically, with insufficient-stock sales rejected.
- Revenue, the best-selling product and category-wise sales can be analyzed and visualized.
- SQLite provides persistent local storage, so products and sales remain available after the program is closed.
- A sale performs both a sales insertion and a stock update, keeping the current inventory synchronized with recorded transactions.
- Input validation and controlled error messages make the application safer to operate with incorrect entries.

Conclusion: Python, OOP, Tkinter, SQLite, Pandas and Matplotlib can be integrated into a practical inventory and sales management application.

Limitations: the application is a local single-user program, has no online or cloud synchronization, and provides only basic analytics. Products cannot yet be edited or deleted. It also has no authentication or role-based permissions, no search, and its analytics are descriptive only (no forecasting).

8. Benefits of this Micro–project

The system offers a simple interface that non-technical users can operate easily, with persistent database storage so that records are not lost when the program is closed. Automatic stock updates and total calculation reduce manual work and calculation errors, while quick revenue, best-seller

and category-wise insights support better stocking decisions. For the team, the project gave hands-on experience in integrating OOP, GUI, database and data-analysis tools.

Total revenue gives a quick summary of sales value, the best-selling product identifies the item with the highest recorded unit demand, and the category-wise chart makes it easy to compare sales contribution across categories. Academically, the project requires an understanding of classes and methods, GUI event handling, SQL queries and commits, Pandas DataFrames and groupby operations, plotting and exception handling, which makes it more comprehensive than a console-only inventory program while keeping it manageable.

9. Area of Future Improvement

The application can be extended with search and filtering, low-stock alerts, and product update and delete options. Monthly sales reports, CSV/PDF export and advanced sales forecasting can be added for deeper analysis. A web-based, multi-user version with cloud synchronization would allow several staff members or branches to use the system together.

For larger-scale deployment, SQLite could be replaced by MySQL or PostgreSQL and the desktop GUI migrated to a web framework such as Flask or Django, which would support centralized access and multiple users. Supplier and purchase modules could track how stock enters the store, and the analytics dashboard could show daily, weekly and monthly trends, top-five products and category contribution percentages. These extensions are future scope and are not part of the current application.

Current Feature	Future Extension	Benefit
SQLite local database	MySQL / PostgreSQL	Multi-user centralized storage
Basic inventory table	Search / filter / update / delete	Better inventory control
Basic analytics	Monthly and trend dashboard	More detailed decision support
Local desktop GUI	Web application	Remote accessibility
Descriptive analysis	Forecasting / reorder logic	Future planning

Table 9: Current features and future extensions

10. References

1. Python Software Foundation. ‘tkinter — Python interface to Tcl/Tk’, Python 3 Documentation. https://docs.python.org/3/library/tkinter.html
2. Python Software Foundation. ‘sqlite3 — DB-API 2.0 interface for SQLite databases’, Python 3 Documentation. https://docs.python.org/3/library/sqlite3.html
3. The pandas development team. ‘pandas Documentation’. https://pandas.pydata.org/docs/
4. Hunter, J. D. (2007) — ‘Matplotlib: A 2D Graphics Environment’, Computing in Science & Engineering, 9(3), pp. 90–95. https://matplotlib.org/stable/
5. Matthes, Eric (2023) — ‘Python Crash Course’, 3rd Edition, No Starch Press, San Francisco.
6. Python Software Foundation. ‘Python Documentation’, Python 3. https://docs.python.org/3/
7. McKinney, Wes. ‘Python for Data Analysis’, O’Reilly Media.
8. Ramalho, Luciano. ‘Fluent Python’, O’Reilly Media.
9. Earnest, Joshua and Rachel, Sthuthi (2024) — ‘Outcome-based and Competency-focused Curriculum for Higher Education’, Viva Books, New Delhi.